Analiza porównawcza wybranych cech języków Java i Rust na podstawie programów realizujących równoważne funkcjonalności

Karol Mac

Opiekun: dr inż. Łukasz Gałka

Politechnika Lubelska · Projekt naukowy – realizacja



Java

vs



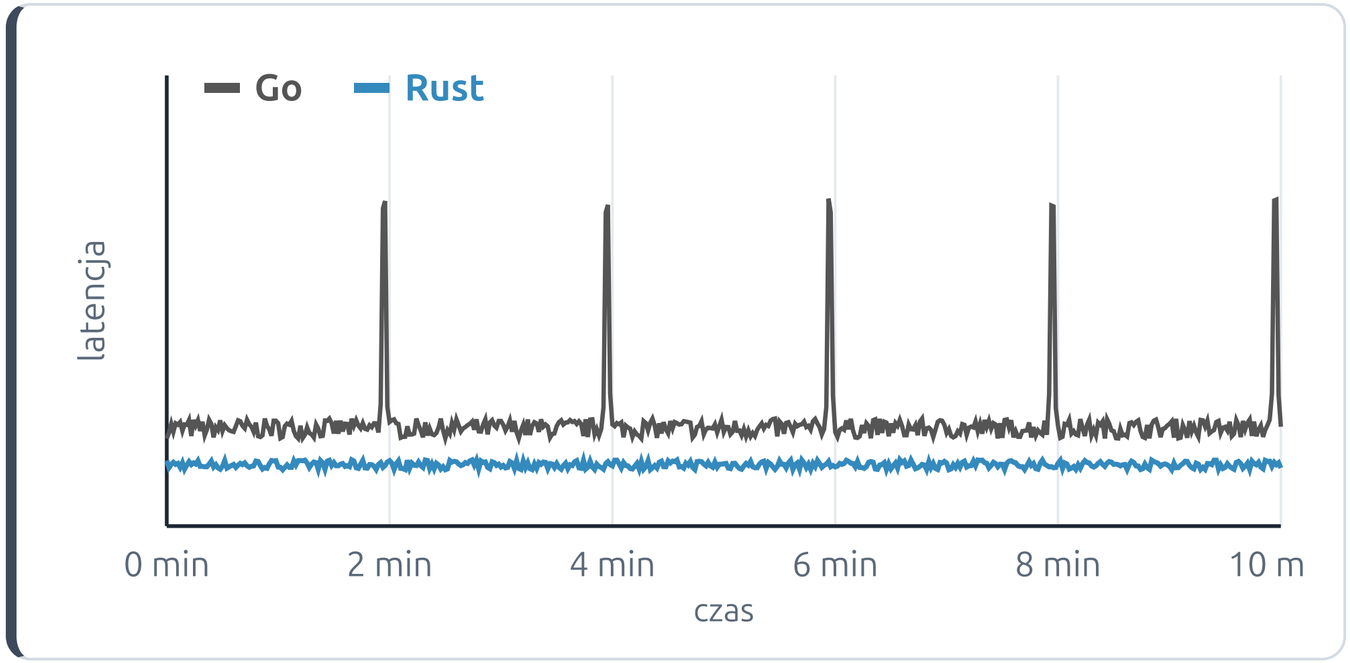
Rust

Kto uważa, że program w Ruście jest zawsze szybszy od tego samego programu w Javie?



podnieś rękę

Discord: z Go na Rust 2020



SERWIS „READ STATES”

GC w Go wymuszany **co 2 min**

Regularne **skoki latencji** i CPU

Rust: **brak skoków**

⚠ To **Go**, nie Java

Czy to samo dotyczy Javy z nowoczesnym G1?

Warunki idealne vs rzeczywistość

BENCHMARK MÓWI		UŻYTKOWNIK / FIRMA ODCZUWA
„średnio”	→	**skrajne** latencje p99, max
czas CPU	→	**rachunek za RAM** kontenery, chmura

Benchmarks Game: średni czas, *cpu secs* · benchmarksgame-team.pages.debian.net
TechEmpower: żądania na sekundę · techempower.com/benchmarks

Cel pracy

PROBLEM

Wybór języka często opiera się na przekonaniach, brakuje porównań **identycznych programów** w różnych obciążeniach

CEL

Zmierzyć różnice Javy 25 i Rusta: **czas, pamięć, wpływ GC, skalowalność** w 4 scenariuszach

Pytania badawcze

1 · czas

Jak duża jest różnica czasu wykonania i od czego zależy?

2 · pamięć

Ile pamięci zajmuje każdy język i jak zmienia się ona w czasie?

3 · GC

Czy pauzy GC są widoczne w latencji, a jeśli tak, to w której części rozkładu?

4 · współbieżność

Jak oba języki skalują się wraz z liczbą wątków?

Zakres pracy



Problem
pytania badawcze

Implementacja
4 scenariusze × 2 języki
tylko biblioteki standardowe

Pipeline
pomiarowy (Python)

Pomiary
czas · RAM · GC · latencja

Analiza
i wnioski

Java vs Rust: dwa modele wykonania

`.java` → javac → bytecode → JVM: interpreter → JIT C1/C2 → G1 GC

kod wykonuje środowisko uruchomieniowe (JVM) · pamięć zwalnia garbage collector

`.rs` → rustc → LLVM → natywny plik wykonywalny → ownership · `Drop`

kompilacja przed uruchomieniem · pamięć zwalniana przy wyjściu ze scope'u

Dlaczego ta para? Dwa skrajne modele zarządzania pamięcią: GC vs ownership, oba w backendzie i systemach

4 scenariusze

S1 · CPU

Mandelbrot 4096×4096 → PPM

JIT vs LLVM

S2 · I/O

CSV 50 mln wierszy (~5 GB), filtr top-1%

parsowanie, alokacje

S3 · współbieżność

Dynamika molekularna LJ, 108 000 cząstek, P wątków + bariera

skalowanie

S4 · pamięć

Strumień 10^9 zdarzeń IoT, z-score + średnia krocząca

RAM, latencja

Dane wejściowe

	Co	Rozmiar	Seed
S1 CPU	4096×4096 px, ≤ 1000 iteracji	brak wejścia	—
S2 I/O	50 mln transakcji, 7 kolumn	~4,7 GB	42
S3 współb.	108 000 / 4 000 cząstek, sieć FCC, prędkości Maxwella-Boltzmanna	5,2 MB / 0,2 MB	42
S4 pamięć	10^9 zdarzeń, 1000 sensorów, 2% anomalii	20 GB	42

S3: parametry jak w benchmarku **LJ LAMMPS**: $\rho^* = 0{,}8442 \cdot T^* = 1{,}44 \cdot r_c = 2{,}5 \cdot \Delta t = 0{,}005$

Projekt eksperymentu

	Java 25	Rust
S1 · CPU	czas RAM GC	czas RAM
S2 · I/O	czas RAM GC	czas RAM
S3 · współb.	czas RAM GC × 7 P	czas RAM × 7 P
S4 · pamięć	czas RAM GC	czas RAM

czas hyperfine: 3 rozgrzewki + 10 pomiarów

RAM RSS co 10 ms, 5 przebiegów

GC log `-Xlog:gc*`, 1 przebieg

P wątki {1, 2, 3, 4, 6, 8, 12} · N = 108k / 4k

WSPÓLNE WARUNKI

- tylko biblioteki standardowe
- kod idiomatyczny
- ten sam algorytm i dane
- domyślne flagi

WALIDACJA POPRAWNOŚCI

- S3: Java ↔ Rust, pozycje < 10^{-6}
- S3: dryf energii < 10^{-3}
- S3: wynik niezależny od P
- S2: identyczne pliki wyjściowe

Metryki

Czas wykonania

mediana z hyperfine

Szczytowy RSS

ile RAM-u proces zabrał systemowi

RAM w czasie

jak pamięć rośnie i wraca do OS

Pauzy GC

ile, jak długie, kiedy

Latencja zdarzenia

p99 = 99% zdarzeń szybszych niż…

Czas kroku

jeden krok symulacji S3

S(P) = T(1) / T(P)

przyspieszenie na P wątkach

E(P) = S(P) / P

efektywność, ideał = 1

RSS, nie heap used: to widzi system operacyjny i za to się płaci

Środowisko i narzędzia

AMD Ryzen 5 7500X3D

6 rdzeni / 12 wątków

32 GB RAM

Windows 11

hyperfine

czas wykonania

OpenJDK 25.0.2

G1 · `-Xms512m -Xmx8g`

Rust 1.98.0

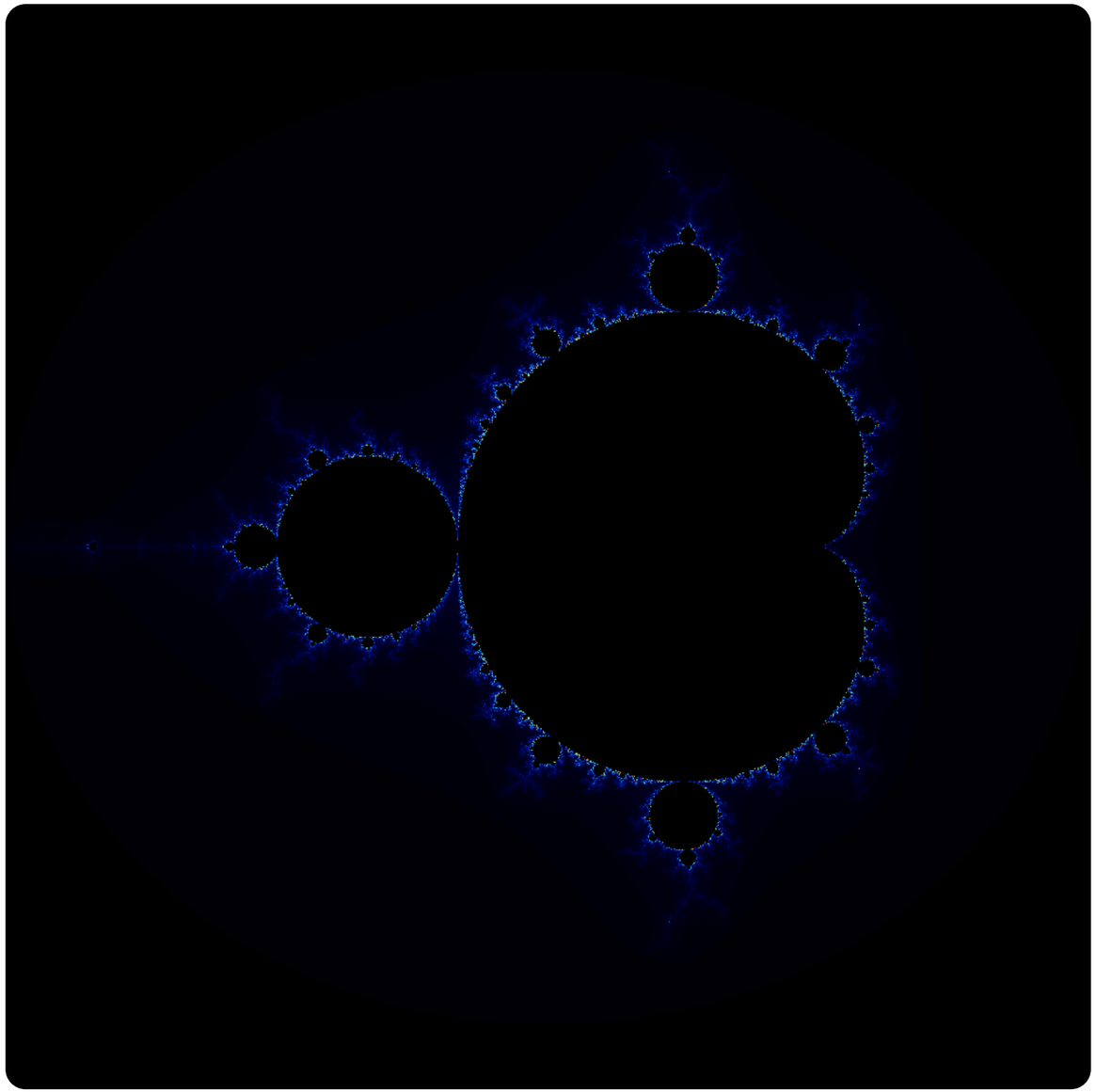
edition 2024

`cargo build --release`

Python

psutil: RSS · numpy: statystyki
matplotlib + seaborn: wykresy

S1 · CPU: zbiór Mandelbrota



4096 × 4096 pikseli

czyste obliczenia
zmiennoprzecinkowe

JIT vs LLVM

Miniatura z tym samym algorytmem i paletą (1000 × 1000)

O ile szybszy jest Rust w liczeniu Mandelbrota?

A

2×

B

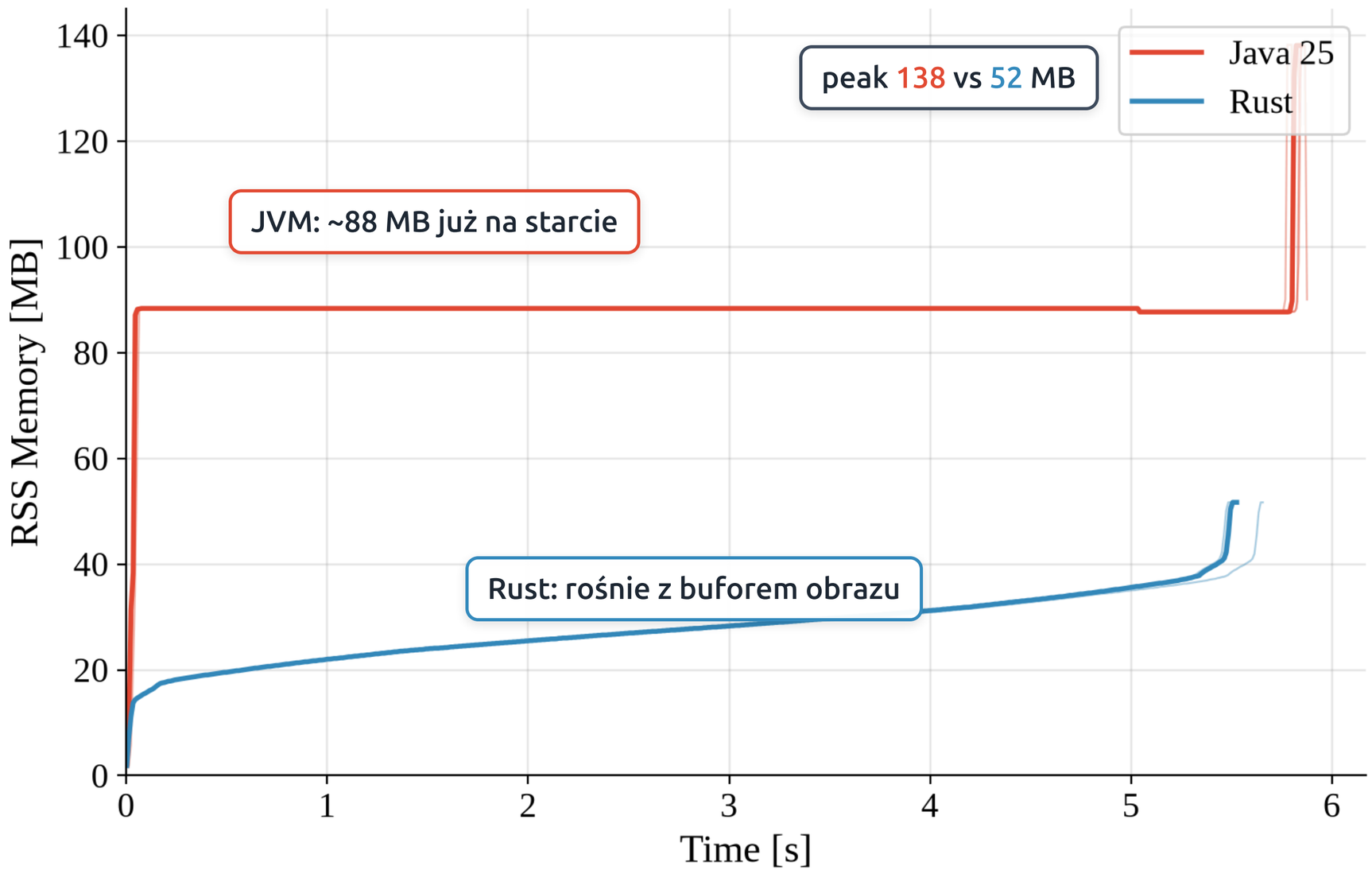
30%

C

6%

S1 · CPU: wynik

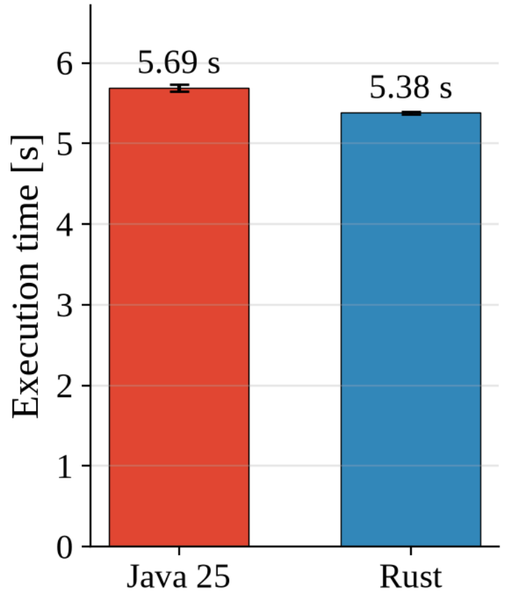
A 2× | B 30% | C 6%



~6%

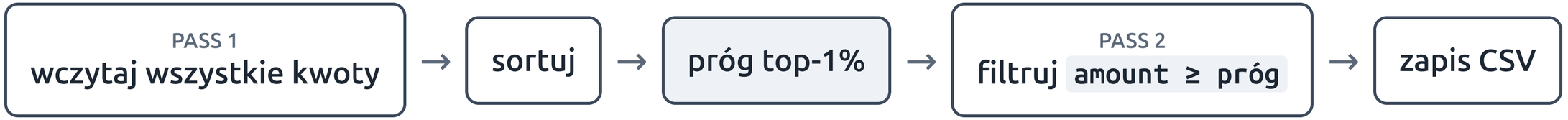
Java ×1,06

5,69 s vs 5,38 s



S2 · I/O: filtrowanie CSV

```
transaction_id,timestamp_ms,amount,sender_id,receiver_id,category,description
1,1700000000664,11133.99,ACC303832645883,ACC245864938384,TRANSFER,Payment ref #98847493
2,1700000000778,67670.27,ACC600832336208,ACC648913461122,INVESTMENT,Payment ref #04265799
3,1700000000818,9370.43,ACC555050052120,ACC028355454532,UTILITY,Payment ref #26687537
… × 50 000 000 wierszy (~5 GB)
```



Ile razy szybciej Rust przefiltruje ~5 GB CSV?

A

1,1×

B

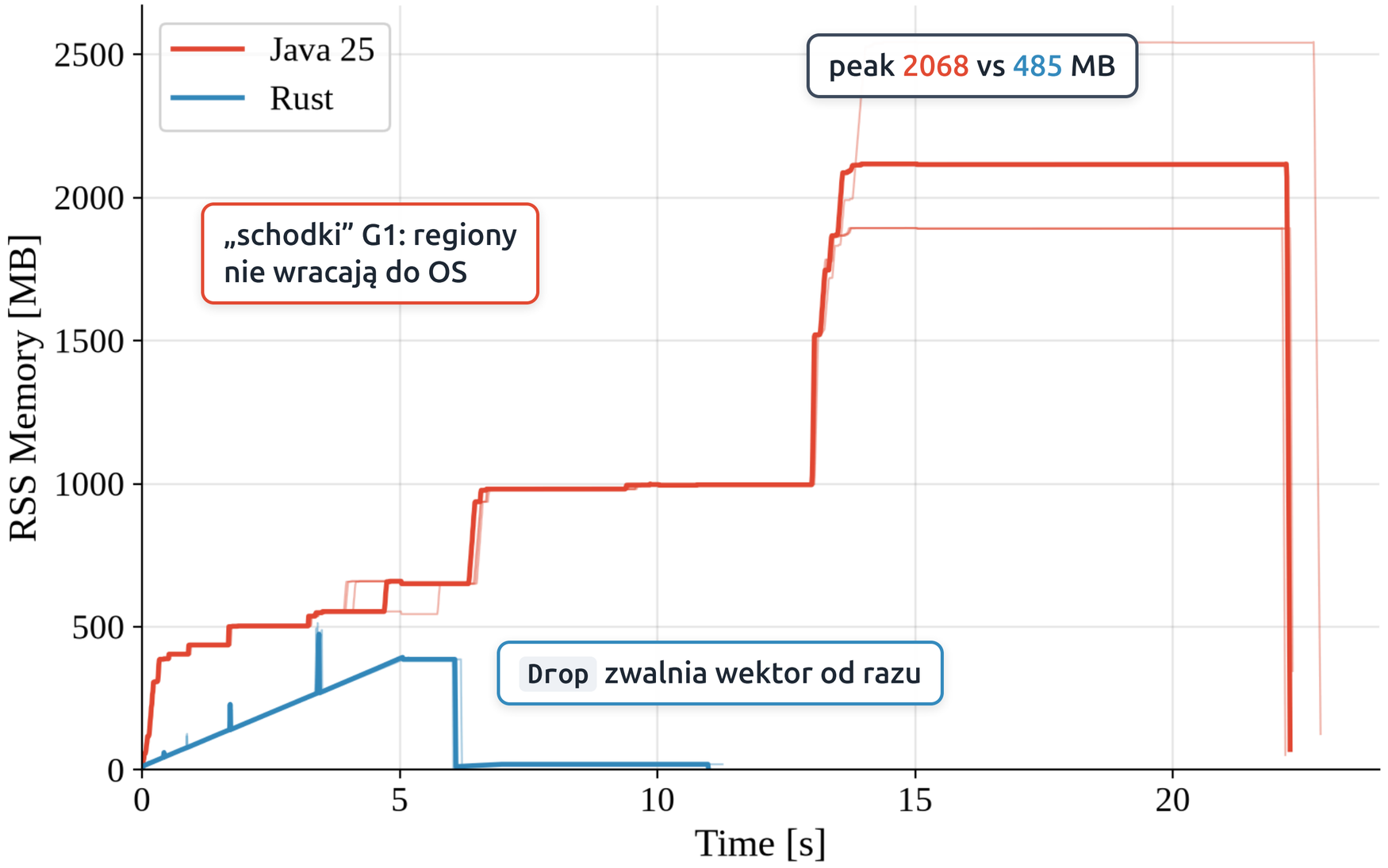
2×

C

10×

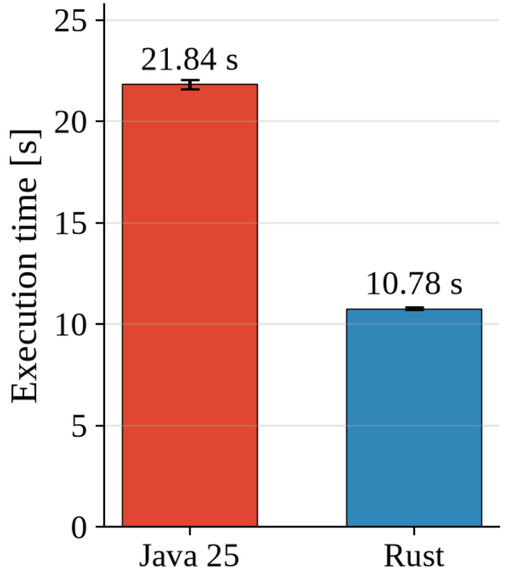
S2 · I/O: wynik

A 1,1× | B 2× | C 10×

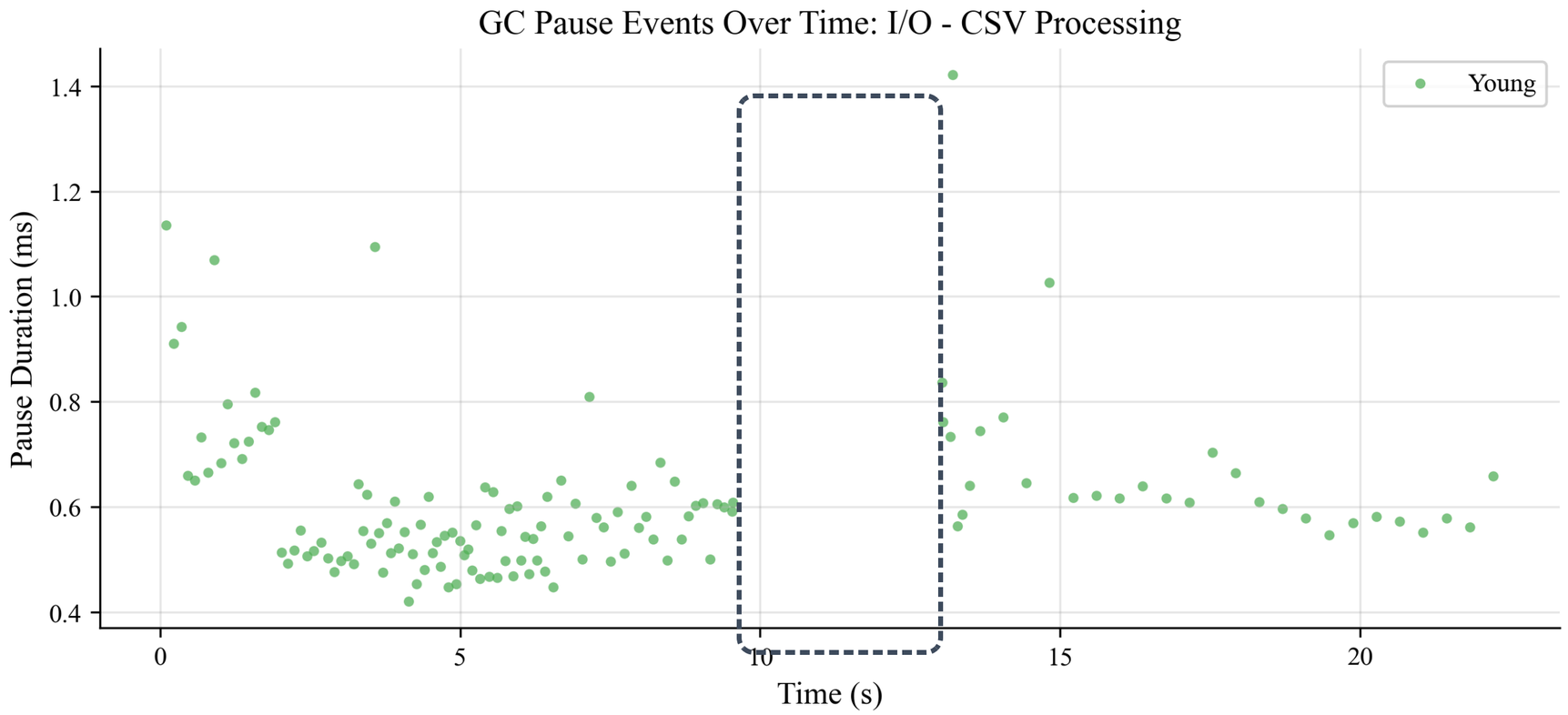


×2,0

21,84 s vs 10,78 s



S2 · I/O: pauzy GC w Javie



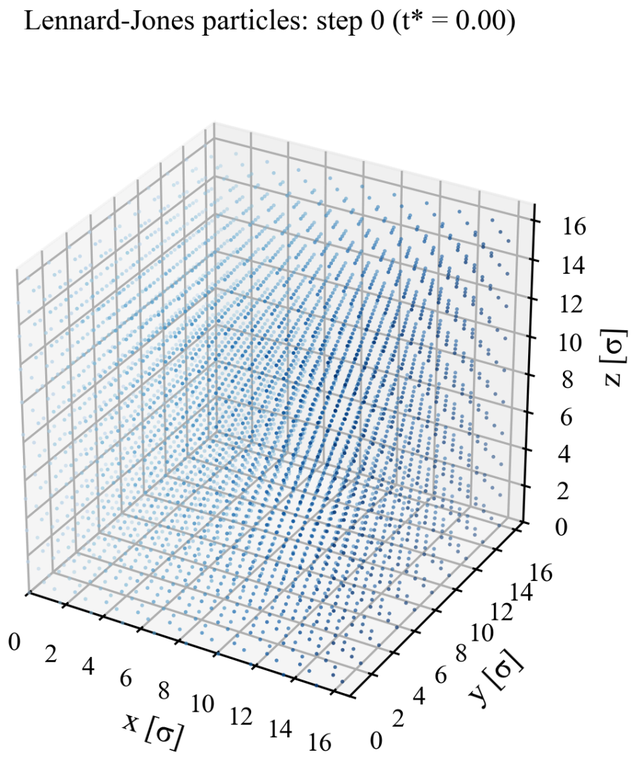
↑ ~3,5 s bez pauz: sortowanie `double[]` (bez alokacji)

134
pauzy (wszystkie Young)

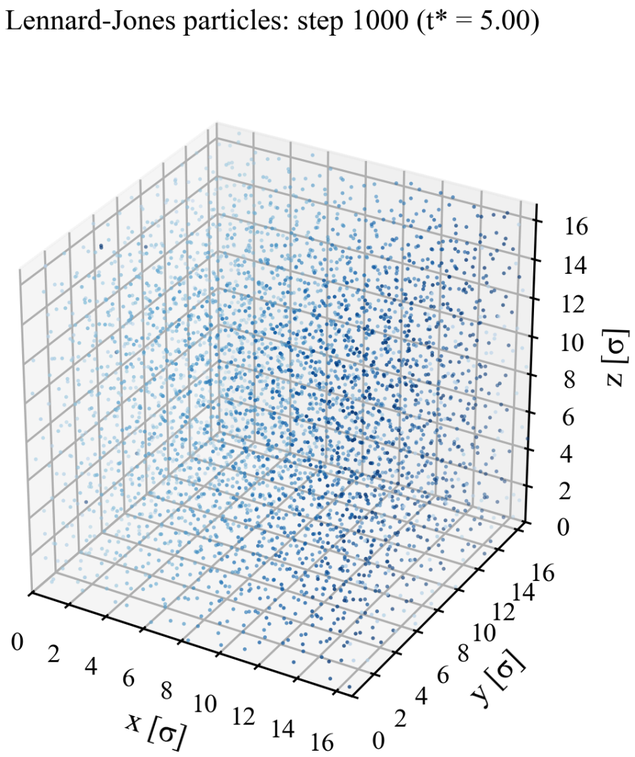
0,58 ms
mediana · maks. 1,42 ms

81 ms
łącznie, ~0,4% czasu

S3 · współbieżność: dynamika molekularna



krok 0: kryształ (FCC)



krok 1000: ciecz

JEDEN KROK

A · pozycje

BARIERA

B · siatka komórek
(szeregowo)

BARIERA

C · siły

BARIERA

D · prędkości

BARIERA

P wątków roboczych

Snapshoty z przebiegu 4 000 cząstek. Pomiary: 108 000 cząstek, 200 kroków.

Który język lepiej skaluje się na 12 wątkach?

A

Java

B

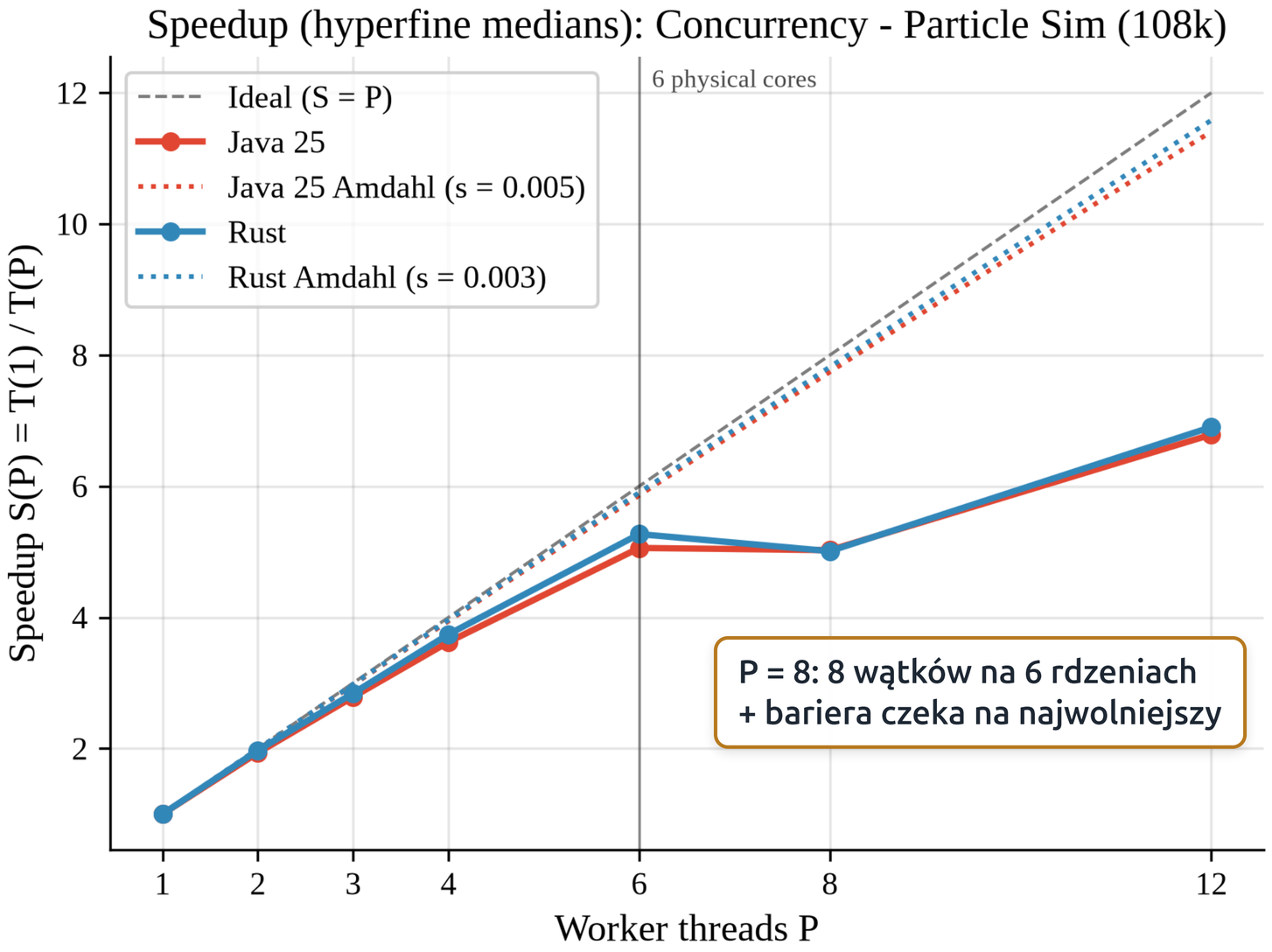
Rust

C

remis

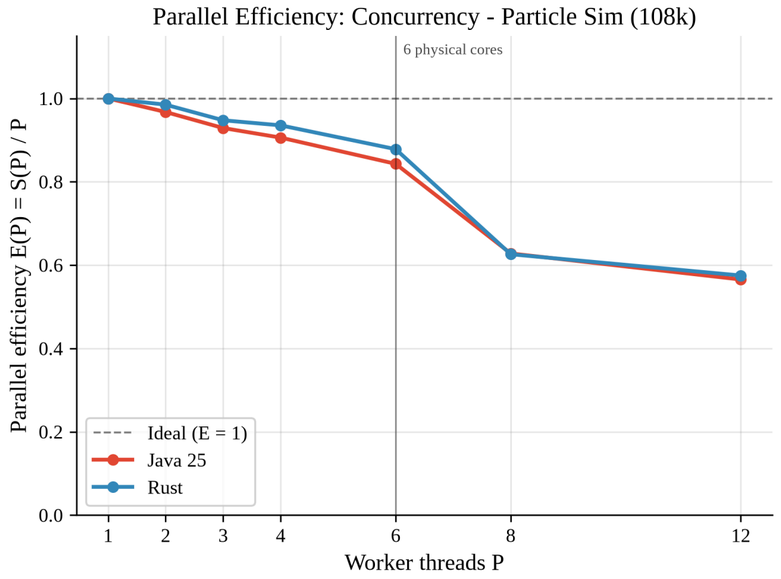
S3 · współbieżność: wynik

A Java | B Rust | C remis

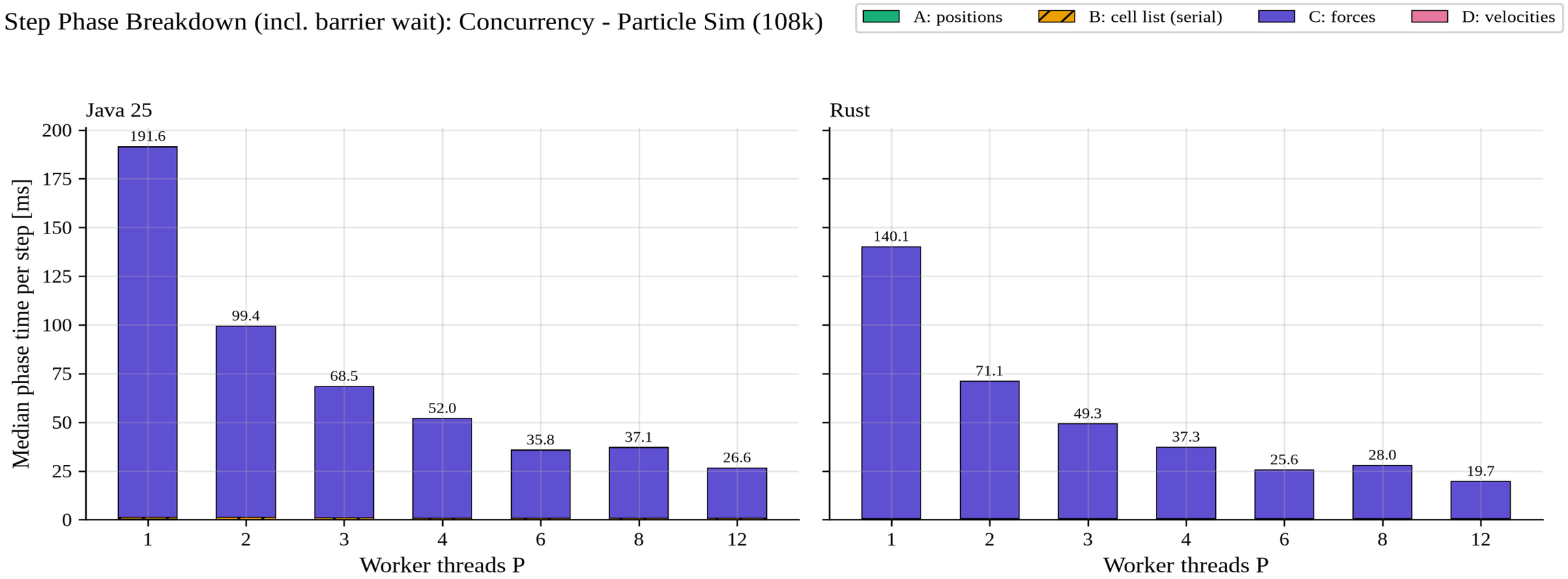


6,8 vs 6,9

S(12)



S3 · gdzie idzie czas kroku?

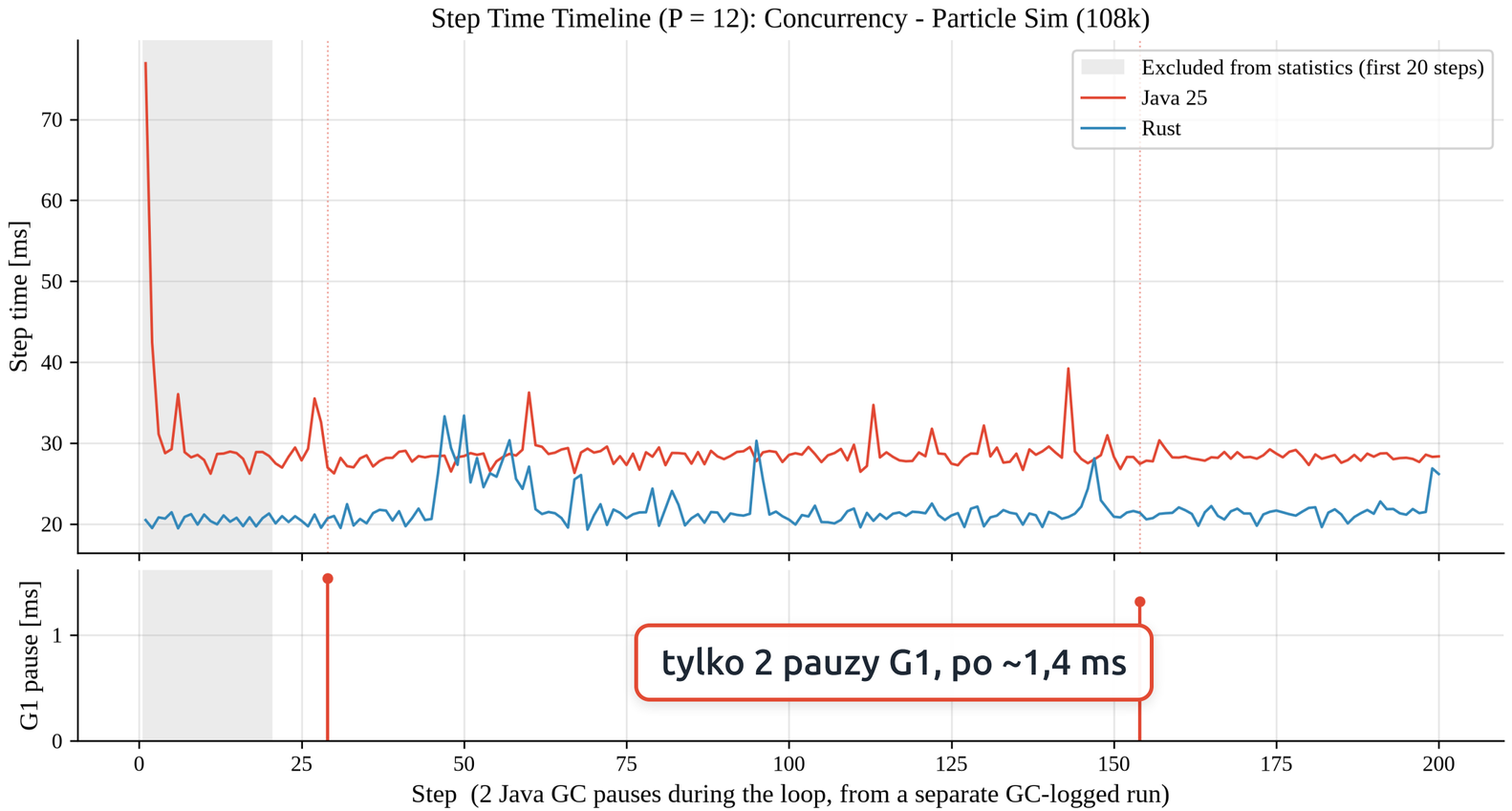


faza sił (C) dominuje: 190 vs 139 ms przy P = 1

część szeregowa (B): **0,3–0,5%**

Rust **~1,4×** szybszy przy każdym P

S3 · czas kroku: czy GC psuje barierę?



CZAS KROKU, P = 12

p50	28,7	21,3 ms
p99	38,1	31,4 ms
p99/p50	1,33	1,48

Przewidywany efekt bariery **nie wystąpił**

S4 · pamięć: strumień zdarzeń IoT

JEDNO ZDARZENIE: REKORD 20 BAJTÓW

timestamp_ns	sensor_id	value
u64 · 8 B	u32 · 4 B	f64 · 8 B

1. **odczyt rekordu** — z pliku 20 GB
2. **stan sensora** — mapa: 1000 sensorów × okno 128 wartości
3. **średnia i σ okna** — liczone krocząco
4. **anomalia?** — |value – średnia| > 3σ

latencja zdarzenia = czas kroków 1–4, mierzona dla **każdego** zdarzenia

10^9 zdarzeń

2% anomalii

Ile razy więcej RAM zużyje Java?

A

1,5×

B

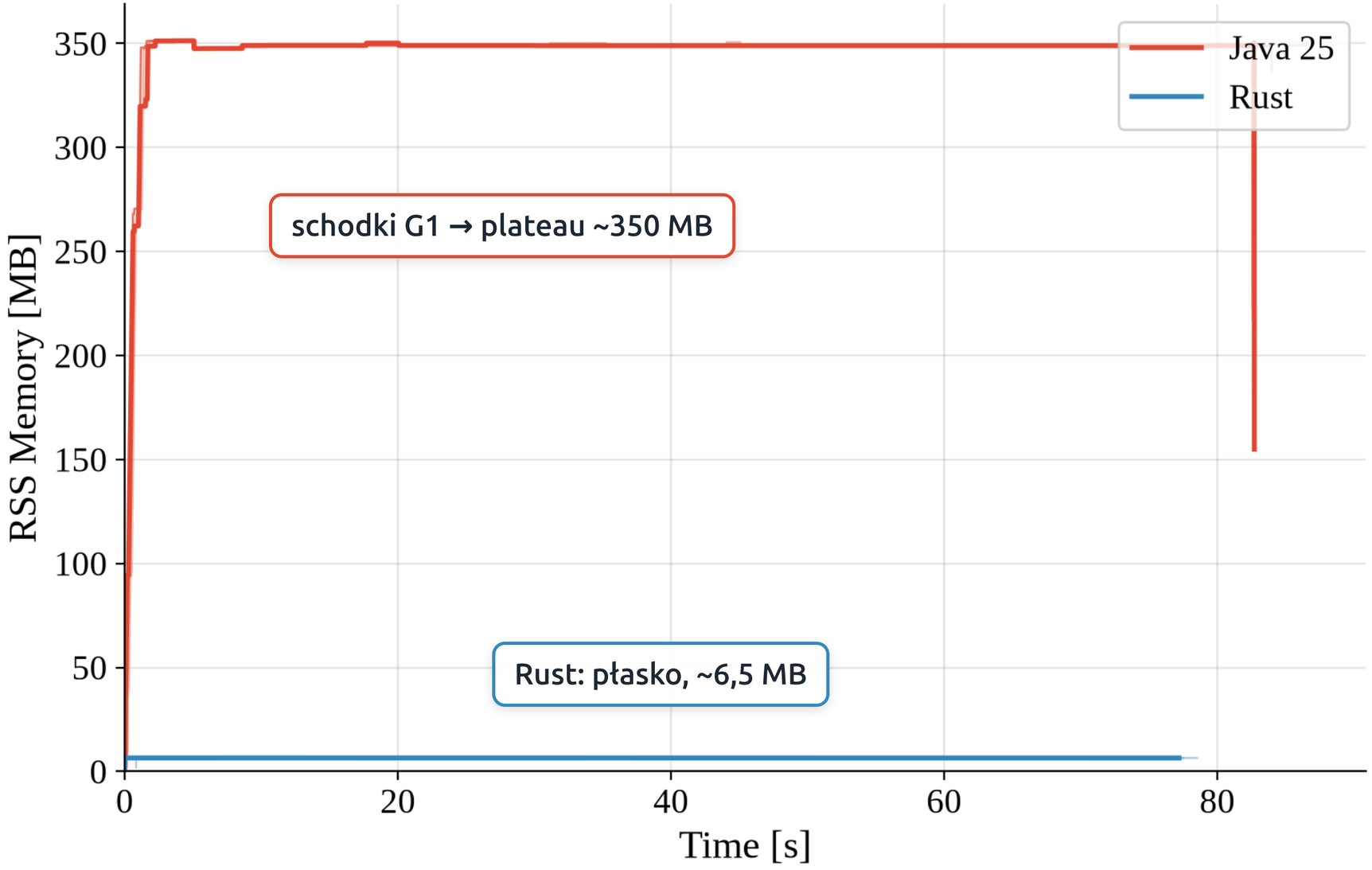
5×

C

50×

S4 · pamięć: wynik

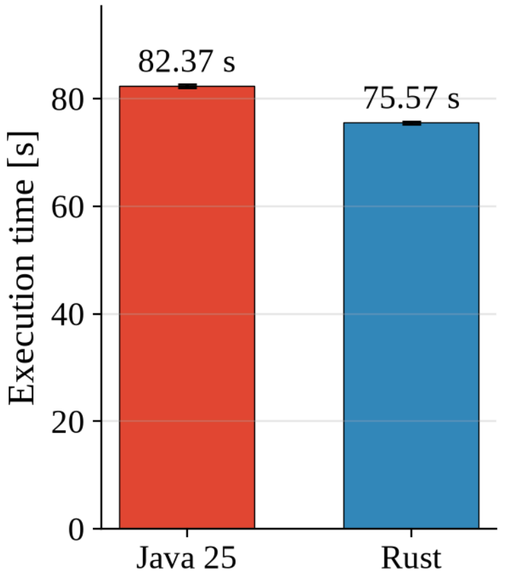
A 1,5× | B 5× | C 50×



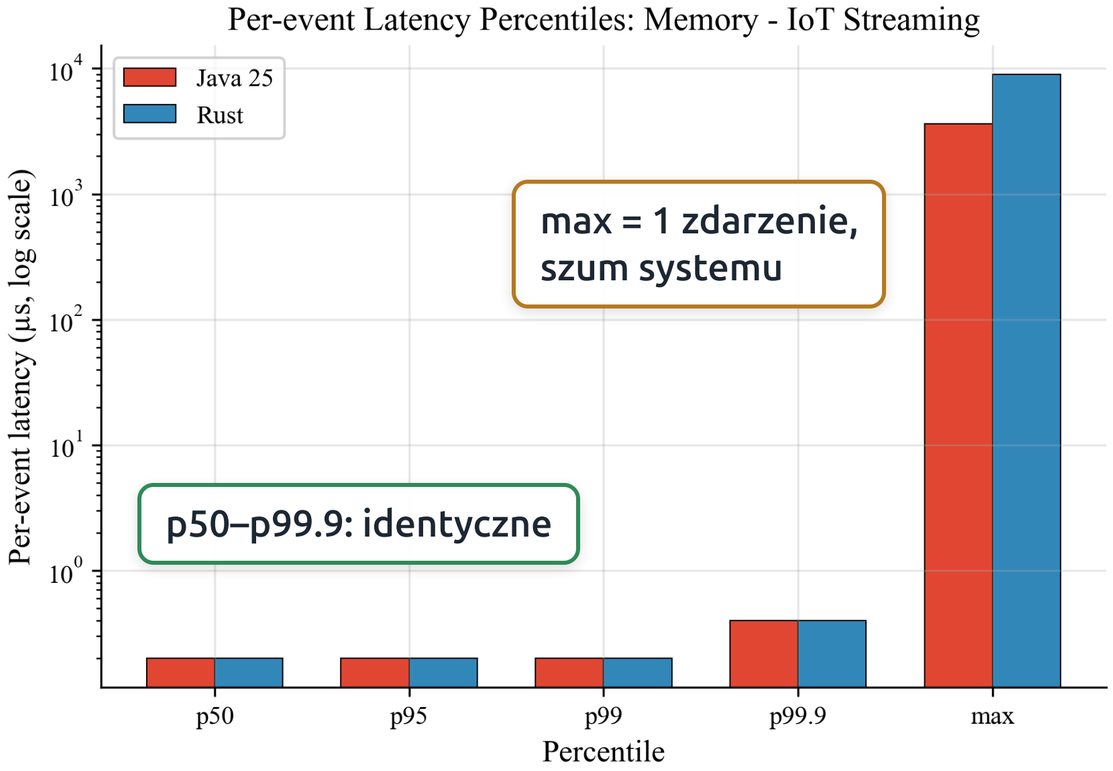
×54

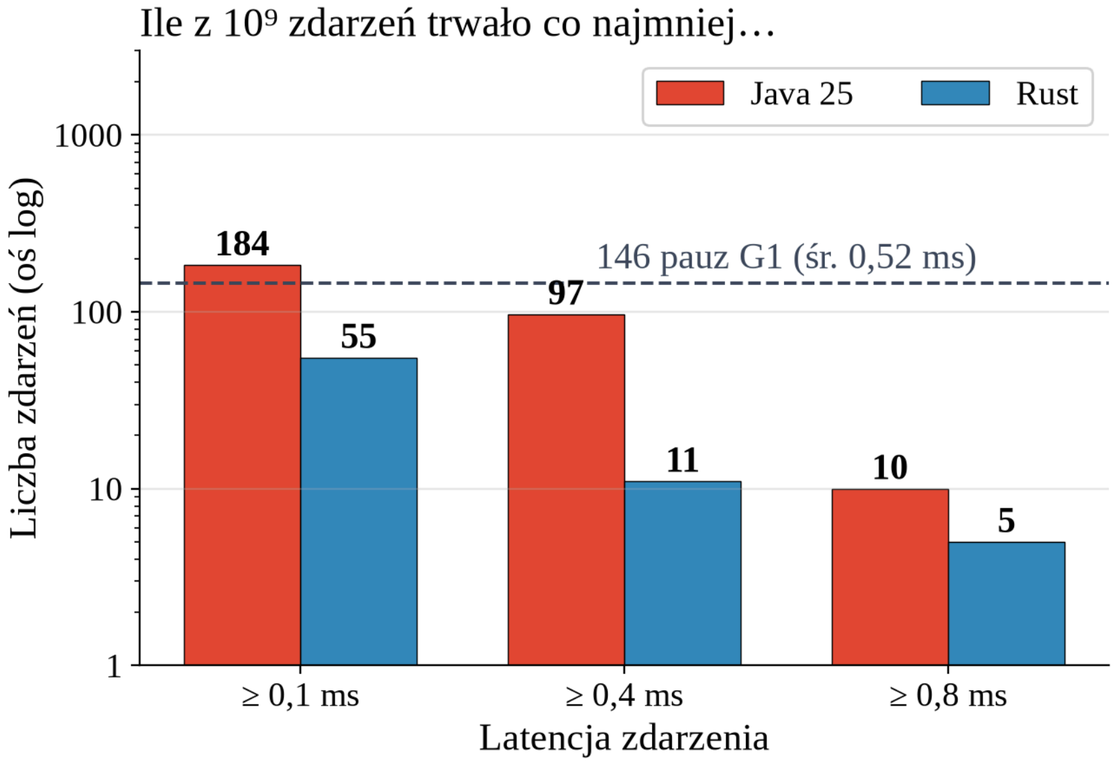
351 MB vs 6,5 MB

czas: Java ×1,09



S4 · latencja: gdzie widać GC?

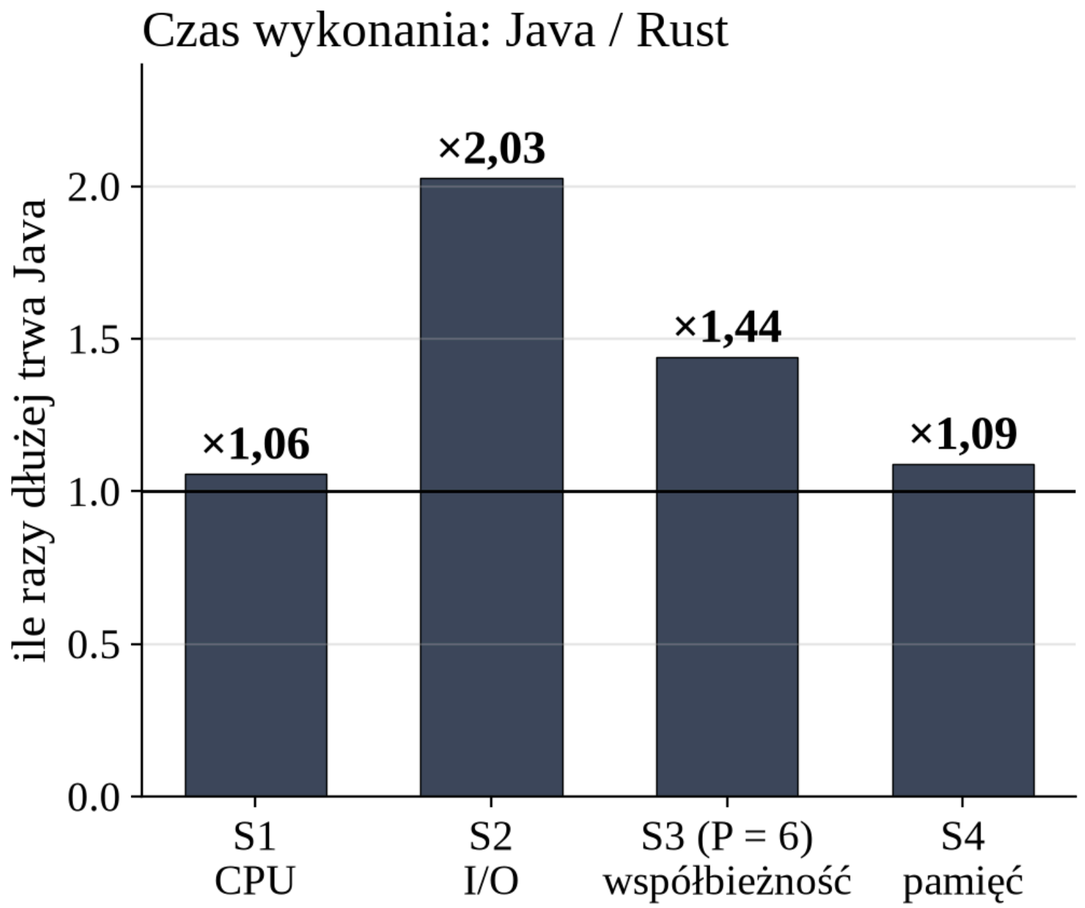


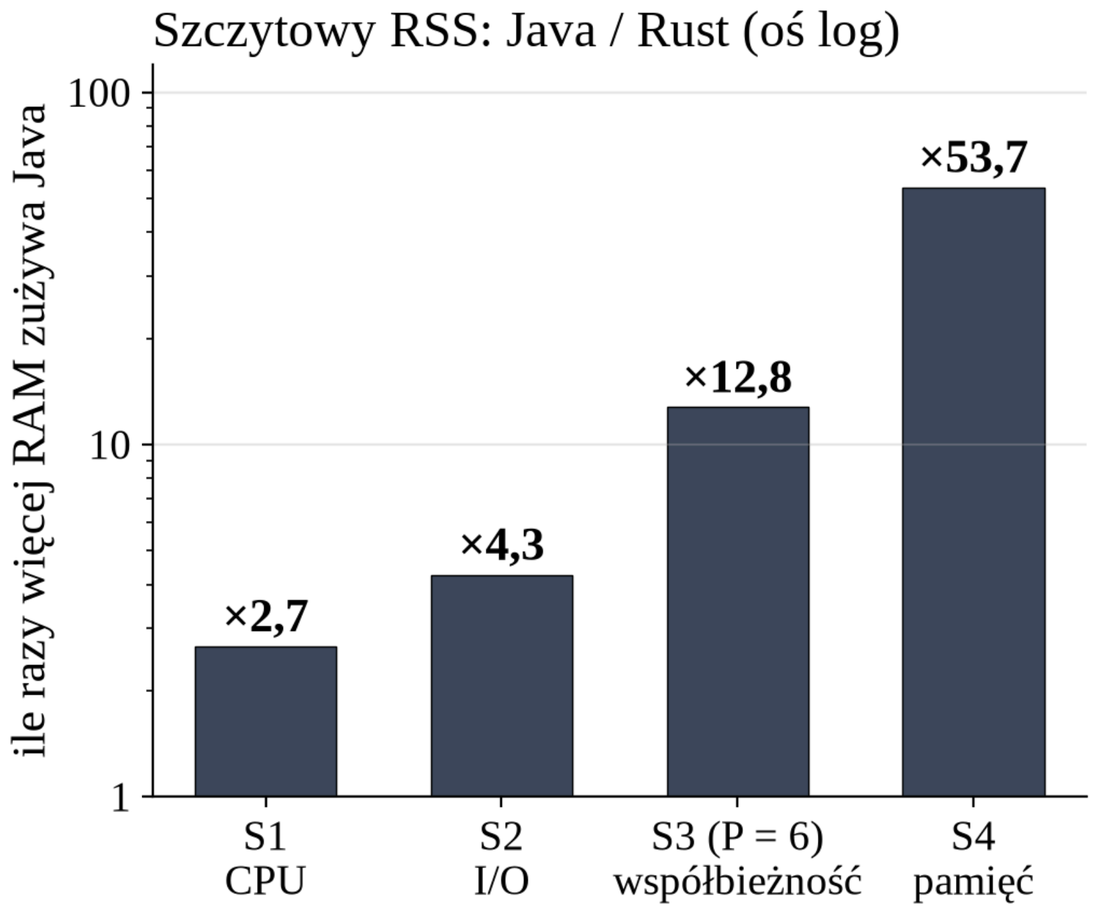


97 vs 11 zdarzeń > 0,4 ms ≈ 146 pauz G1 po ~0,5 ms

GC kosztuje, ale to nie Discord

Zbiorczo: 4 scenariusze





Java / Rust: mediana hyperfine (czas) i średni szczytowy RSS z 5 przebiegów; S3 przy P = 6

Odpowiedzi

1 · czas	Rust szybszy wszędzie: S1 ×1,06 · S4 ×1,09 · S3 ×1,38–1,44 · S2 ×2,03
2 · pamięć	RSS_{Java} / RSS_{Rust}: S1 ×2,7 · S2 ×4,3 · S3 (P = 6) ×12,8 · S4 ×54
3 · GC	p50–p99.9 identyczne; skrajnie 97 vs 11 zdarzeń > 0,4 ms. **Efekt mały**
4 · współb.	Skalowanie prawie identyczne: E(6) = 84–88% · $\lvert\Delta S(P)\rvert \le 4\%$ · S(12): 6,8 vs 6,9

Wnioski

Rust zawsze szybszy? **Tak, ale od 6% do ×2.** Za to pamięć: **od ×2,7 do ×54**

Rust, gdy…

- liczy się pamięć: kontenery, embedded, krótkie CLI
- masowe I/O z parsowaniem
- przewidywalny czas wykonania

Java, gdy…

- dominują obliczenia: JIT traci do kodu natywnego tylko ~6%
- liczy się ekosystem i koszt zespołu
- opóźnienie GC realne, ale małe (~0,5 ms)

Współbieżność

- skalowanie identyczne
- język zmienia tylko punkt startu

Ograniczenia: 1 maszyna, Windows (zegar 100 ns, szum schedulera) domyślne flagi GC kod idiomatyczny, nieoptymalizowany

Źródła

1. J. Howarth, „Why Discord is switching from Go to Rust”, Discord Blog, 4.02.2020. discord.com/blog/why-discord-is-switching-from-go-to-rust
2. D. Detlefs, C. Flood, S. Heller, T. Printezis, „Garbage-First Garbage Collection”, ISMM ’04, ACM, 2004, s. 37–48. doi.org/10.1145/1029873.1029879
3. G. M. Amdahl, „Validity of the single processor approach to achieving large scale computing capabilities”, AFIPS ’67, 1967, s. 483–485. doi.org/10.1145/1465482.1465560
4. A. P. Thompson i in., „LAMMPS – a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales”, Computer Physics Communications 271, 108171, 2022. doi.org/10.1016/j.cpc.2021.108171
5. LAMMPS, benchmark LJ (`bench/in.lj`). github.com/lammps/lammps/tree/develop/bench
6. S. Klabnik, C. Nichols, C. Krycho, „The Rust Programming Language”. doc.rust-lang.org/book
7. Oracle, „Garbage-First (G1) Garbage Collector”, HotSpot VM GC Tuning Guide, Release 25. docs.oracle.com/en/java/javase/25/gctuning
8. OpenJDK, „JEP 248: Make G1 the Default Garbage Collector”. openjdk.org/jeps/248
9. D. Peter, „hyperfine: A command-line benchmarking tool”. github.com/sharkdp/hyperfine
10. G. Rodola, „psutil”. github.com/giampaolo/psutil
11. J. D. Hunter, „Matplotlib: A 2D Graphics Environment”, Computing in Science & Engineering 9(3), 2007. doi.org/10.1109/MCSE.2007.55
12. M. L. Waskom, „seaborn: statistical data visualization”, JOSS 6(60), 3021, 2021. doi.org/10.21105/joss.03021
13. C. R. Harris i in., „Array programming with NumPy”, Nature 585, 357–362, 2020. doi.org/10.1038/s41586-020-2649-2
14. The Computer Language Benchmarks Game. benchmarksgame-team.pages.debian.net/benchmarksgame · TechEmpower, „Web Framework Benchmarks”. techempower.com/benchmarks
15. OpenJDK, „JDK 25”. openjdk.org/projects/jdk/25 · Rust Project, „Rust”. rust-lang.org

Dziękuję za uwagę

Analiza porównawcza wybranych cech języków Java i Rust na podstawie programów realizujących równoważne funkcjonalności

Karol Mac

Opiekun: dr inż. Łukasz Gałka

Wszystkie liczby

	S1 CPU	S2 I/O	S3 (108k, P = 6)	S4 IoT
Czas Java [s]	5,69	21,84	7,72 (P=1: 39,07)	82,37
Czas Rust [s]	5,38	10,78	5,36 (P=1: 28,24)	75,57
Czas Java / Rust	×1,06	×2,03	×1,44 (P=1: ×1,38)	×1,09
Peak RSS Java [MB]	138	2068	294	351
Peak RSS Rust [MB]	51,7	485	23,1	6,5
RSS Java / Rust	×2,7	×4,3	×12,8	×53,7
Pauzy GC (Java)	0	134 · med. 0,58 ms	3 · śr. 1,83 ms	146 · śr. 0,52 ms
Inne	—	—	krok p50: 39,3 / 27,5 ms p99: 51,1 / 37,6 ms	p50 = p99 = 0,2 µs p99.9 = 0,4 µs (oba)

Czas: mediana hyperfine (10 pomiarów). RSS: średnia szczytów z 5 przebiegów, MB = 1024^2 B (jak na wykresach).